# Gestor de identidades y accesos

*Informe de implementación y benchmark*

**Materia:** Laboratorio 2

**Integrante identificado:** Ignacio Piquín Villegas

**Integrantes restantes:** pendiente completar los nombres del grupo

**Fecha:** 28 de octubre de 2026

**Resumen**

Este trabajo modela un sistema de gestión de identidades y accesos para una organización. El sistema administra empleados, áreas, puestos, roles, permisos sobre aplicaciones, solicitudes de acceso, aprobaciones y auditoría. Se implementó una alternativa relacional en Oracle APEX y una alternativa documental en MongoDB.

Las dos implementaciones ejecutaron las operaciones principales y una prueba de rendimiento con 5.000 empleados y 5.000 solicitudes sintéticas en cada motor. Ambas devolvieron 12.500 combinaciones de permisos efectivos y 7.500 filas de detalle de solicitudes. En las cinco ejecuciones, la mediana informada por Oracle APEX fue menor para las dos consultas. Como Oracle se ejecutó en un servicio hospedado y MongoDB en una máquina local, y cada entorno mide el tiempo de manera diferente, el resultado describe esta prueba y no demuestra que un motor sea más rápido en general.

**Caso propuesto**

Una organización necesita controlar qué sistemas y acciones puede utilizar cada empleado. El acceso puede surgir del puesto que ocupa o de una asignación directa. Para solicitar un rol, el empleado registra el motivo y una persona con el rol aprobador correspondiente decide aprobar o rechazar la solicitud. El sistema debe conservar quién solicitó, quién aprobó, cuándo se tomó la decisión y qué cambió.

El problema requiere resolver estas necesidades:

- Mantener los datos de empleados, áreas y puestos.
- Asociar roles a puestos y empleados.
- Definir qué permisos concede cada rol en cada aplicación.

- Registrar solicitudes y sus estados.
- Resolver aprobaciones en uno o más pasos.
- Conservar un historial de eventos para auditoría.
- Consultar los accesos vigentes y las solicitudes pendientes.

**Modelo relacional en Oracle**

La implementación relacional conserva cada entidad compartida una sola vez y la conecta mediante claves primarias y foráneas.

| Entidad o tabla | Responsabilidad |
|---|---|
| AREA | Catálogo de áreas organizacionales. |
| PUESTO | Catálogo de puestos. |
| EMPLEADO | Persona, área y puesto actual. |
| ROL | Catálogo de perfiles de acceso. |
| APLICACION | Sistemas a los que se administran permisos. |
| PERMISO | Acciones posibles dentro de una aplicación. |
| PUESTO_ROL | Roles heredados automáticamente por el puesto. |
| ROL_PERMISO | Permisos incluidos en cada rol. |
| EMPLEADO_ROL | Asignaciones directas o concedidas por solicitudes. |
| SOLICITUD | Pedido de alta, baja o modificación de un rol. |
| APROBACION | Pasos, responsables y decisiones de aprobación. |
| LOG_AUDITORIA | Eventos asociados a usuarios, solicitudes y aprobaciones. |

Las relaciones principales son: un área tiene muchos empleados; un puesto puede tener muchos empleados; puestos y roles se relacionan muchos a muchos mediante PUESTO_ROL; roles y permisos se relacionan muchos a muchos mediante ROL_PERMISO; un permiso pertenece a una aplicación; un empleado puede tener asignaciones directas en EMPLEADO_ROL; un empleado puede generar solicitudes; cada solicitud puede tener varios pasos de aprobación y eventos de auditoría.

El modelo incorpora restricciones de clave, valores permitidos, fechas y referencias. Por ejemplo, las solicitudes distinguen alta, baja y modificación; una modificación debe indicar un rol anterior diferente del nuevo, y una aprobación resuelta registra aprobador y fecha de decisión. La autorización del aprobador y la prohibición de autoaprobación se validan en la operación de negocio, porque dependen de varias tablas.

La consulta de accesos efectivos combina los roles heredados del puesto con los roles directos vigentes y los expande hasta sus permisos. La consulta de una solicitud integra varias tablas mediante JOIN.

### Modelo documental en MongoDB

MongoDB organiza la información en dos agregados raíz:

| Colección | Datos embebidos | Consulta principal |
|---|---|---|
| empleados | Persona, área, puesto, roles del puesto, roles directos, aplicaciones y permisos. | Leer el perfil y calcular los accesos vigentes del empleado. |
| solicitudes | Solicitante, rol solicitado y sus permisos, responsable de aprobación, pasos y eventos de auditoría. | Revisar o resolver una solicitud con su contexto completo. |

Área, puesto, rol, aplicación y permiso se guardan como subdocumentos dentro del agregado que los necesita. La solicitud incluye el perfil del solicitante y del responsable; no guarda solamente sus identificadores para completar esos datos desde otra colección. Las consultas de la implementación no usan $lookup ni colecciones de catálogo.

Los validadores $jsonSchema controlan la forma y los tipos de los documentos. Las reglas que dependen de información duplicada entre snapshots deben mantenerse en las operaciones de actualización. Los documentos raíz tienen _id; los datos relacionados se almacenan completos dentro del agregado.

Los scripts con extensión .js son instrucciones para mongosh, la consola de MongoDB. No se construyó una aplicación JavaScript ni se usó un driver de aplicación. Los comandos de carga, consulta y modificación se ejecutan directamente contra MongoDB.

### Implementación y verificación funcional

### Oracle APEX

Las ejecuciones iniciales en SQL Workshop y la verificación del benchmark se registraron en la tabla. Los scripts 02 y 03 recibieron ampliaciones posteriores, cuya carga y ejecución se deben validar en APEX sobre un esquema limpio o revisando los IDs antes de insertar. La estructura y el benchmark ya medidos no necesitan repetirse.

| Script | Resultado observado |
|---|---|
| 01_modelo_iam.sql | 16 sentencias exitosas y 0 errores; se crearon las tablas y comentarios del modelo. |
| 02_insercion_datos_iam.sql | Ejecución inicial: dos empleados, catálogos y dos solicitudes en aprobación. La versión actual incorpora además los casos de modificación aprobada, baja rechazada y alta para probar el paquete, junto con una asignación directa; las inserciones nuevas requieren validación puntual en APEX. |
| 03_operaciones_iam.sql | Ejecución inicial: aprobación de una solicitud y consulta de permisos con 3 resultados. La versión actual suma cursor, función, procedimiento con control de errores y paquete; estos bloques agregados requieren validación puntual en APEX. |
| 04_datos_benchmark_5000.sql | Se insertaron 5.000 filas en cada tabla de empleados, asignaciones directas, solicitudes y aprobaciones. |
| 05_benchmark_consultas.sql | Se ejecutaron cinco veces las dos consultas de lectura para medir sus tiempos. |

La verificación de cardinalidad en SQL Commands devolvió 5.000 empleados sintéticos, 5.000 solicitudes sintéticas, 12.500 permisos efectivos y 7.500 filas de detalle de solicitudes. Los IDs de demo 1 a 5 son distintos de los rangos sintéticos del benchmark. Por eso las ampliaciones no alteran los resultados de rendimiento; no hace falta repetir la carga ni las consultas del benchmark. En un esquema limpio, ejecutar los scripts en orden. En el esquema ya cargado, no repetir las cargas con identificadores fijos: revisar primero los IDs demo 3 a 5 y probar puntualmente las inserciones y los objetos PL/SQL agregados.

**MongoDB**

Se verificó la carga y operación en MongoDB 8.3.7 con mongosh 2.9.2 sobre Windows x64. El script de estructura creó o actualizó las colecciones y validadores. La carga demo, la aprobación de una solicitud, la actualización del agregado de empleado, la consulta de pendientes, la consulta de accesos y el registro de auditoría se ejecutaron correctamente. La carga sintética agregó 5.000 empleados y 5.000 solicitudes con los catálogos y las relaciones embebidos.

**Benchmark de rendimiento**

Se midieron dos consultas funcionalmente equivalentes:

1. **Accesos efectivos:** combinar roles del puesto y asignaciones directas, y contar las combinaciones distintas de empleado, rol, aplicación y permiso.
2. **Solicitudes pendientes:** recorrer las aprobaciones pendientes y contar las combinaciones de solicitud, aplicación y permiso del rol solicitado, considerando activos el solicitante, el rol aprobador, el rol pedido, la aplicación y el permiso.

El conjunto sintético tiene dos perfiles de igual tamaño. Cada empleado recibe un rol por puesto y uno directo. La mitad produce tres permisos y la otra mitad dos, para un total esperado de 12.500. Cada solicitud tiene una aprobación pendiente; la mitad solicita un rol con dos permisos y la otra mitad un rol con uno, para un total esperado de 7.500. Ambos motores devolvieron esos conteos.

En Oracle APEX se registraron cinco ejecuciones por consulta. APEX presenta ELAPSED en segundos con dos decimales. En MongoDB se cronometró cada operación desde aggregate() hasta recibir toArray() en mongosh; las duraciones se informan en milisegundos enteros.

| Consulta | Filas | Oracle: mediana (rango) | MongoDB: mediana (rango) |
|---|---|---|---|
| Accesos efectivos | 12.500 | 30 ms (20-290) | 203 ms (172-216) |
| Solicitudes pendientes | 7.500 | 10 ms (10-130) | 110 ms (107-117) |

Mediciones de cada repetición: accesos efectivos, Oracle 290, 140, 20, 30 y 20 ms; MongoDB 172, 189, 216, 211 y 203 ms. Solicitudes pendientes, Oracle 130, 100, 10, 10 y 10 ms; MongoDB 109, 113, 107, 117 y 110 ms.

Las medianas observadas en Oracle son menores. Las dos primeras ejecuciones de cada consulta Oracle fueron más lentas que las siguientes, mientras que el intervalo de MongoDB fue más estrecho. En la primera consulta los intervalos se superponen entre 172 y 216 ms; en la segunda, entre 107 y 117 ms. Por ello se informan las cinco mediciones y no solamente una cifra de velocidad.

**Límites de interpretación**

Oracle APEX es un servicio hospedado compartido; MongoDB corrió localmente en el equipo. No se igualaron CPU, memoria, almacenamiento, carga concurrente ni ubicación de red. Además, APEX informa tiempo de sentencia redondeado a centésimas de segundo, mientras que la medición de MongoDB con Date.now() incluye el viaje de ida y vuelta entre mongosh y mongod por loopback. Se hicieron cinco repeticiones, con datos sintéticos y consultas pequeñas. Estos factores impiden atribuir la diferencia al motor por sí solo.

La prueba no crea índices secundarios manuales. Oracle mantiene los mecanismos internos asociados a las restricciones de claves; MongoDB mantiene su índice automático de _id.

### Justificación de la implementación MongoDB

Se eligió MongoDB para la implementación documental porque el caso consulta el perfil y los accesos efectivos de un empleado en conjunto, y revisa cada solicitud junto con el solicitante, el rol solicitado, las aprobaciones y la auditoría. Los dos agregados reflejan esas unidades de lectura y actualización. Además, la consigna de la cátedra pide priorizar documentos embebidos en MongoDB y evitar completar datos mediante referencias entre colecciones.

La estructura relacional Oracle se mantiene como implementación de referencia para ejecutar las mismas operaciones con entidades y restricciones declarativas. El benchmark reporta tiempos medidos de ambas implementaciones; la justificación del modelo no se usa para decidir cuál motor tuvo mejor rendimiento.

### Conclusiones

El caso propuesto queda implementado en Oracle y MongoDB. El modelo Oracle muestra tablas, relaciones y restricciones para controlar integridad. El modelo MongoDB sigue la pauta de documentos embebidos: empleados y solicitudes contienen el contexto relacionado que requieren sus lecturas y operaciones.

Se ejecutaron cargas y consultas en ambos motores con la misma escala sintética. Los conteos coinciden, y se registraron cinco tiempos por consulta en cada plataforma. El resultado observado favorece las medianas de Oracle en este experimento, aunque la diferencia entre entornos y relojes limita la conclusión.

### Archivos de implementación y reproducción

El repositorio adjunta el código fuente SQL y los scripts de mongosh:

- 01_modelo_iam.sql, 02_insercion_datos_iam.sql y 03_operaciones_iam.sql: esquema y operaciones Oracle.
- 04_datos_benchmark_5000.sql y 05_benchmark_consultas.sql: carga y consultas para Oracle.
- mongodb/01_crear_estructura.js, 02_cargar_datos_demo.js y 03_operaciones.js: estructura y operaciones MongoDB.

- mongodb/04_datos_benchmark.js y 05_benchmark_consultas.js: carga sintética y consultas de benchmark.
- mongodb/benchmarking.md: mediciones detalladas y procedimiento de repetición.
- codigo_entrega.txt: consolidación de los scripts para adjuntar como archivo de código.

En un esquema Oracle limpio, ejecutar los scripts 01 a 05 en orden; las cargas demo y benchmark usan identificadores fijos y se ejecutan una sola vez. En un esquema que ya contiene datos, no repetir esas cargas completas. En MongoDB, seguir el orden de ejecución documentado en mongodb/README.md.

**Referencias**


- MongoDB, Enforce Data Consistency with Embedding: https://www.mongodb.com/docs/manual/data-modeling/enforce-consistency/embed-data/.
- MongoDB, Specify JSON Schema Validation: https://www.mongodb.com/docs/manual/core/schema-validation/specify-json-schema/.
- Oracle APEX, Using the Command Editor: https://docs.oracle.com/en/database/oracle/apex/26.2/aeutl/using-command-editor.html.
- Oracle APEX, Using SQL Scripts: https://docs.oracle.com/en/database/oracle/apex/26.2/aeutl/using-sql-scripts.html.